NLP

A Developer's Study Notebook

Concepts · Examples · Projects · Evaluation

Prepared by: Hammad Yousuf

20 Pages

Hammad Yousuf

WITHHAMMAD

1. What NLP does

Language becomes a computational task

1.1 Start with an outcome

- Route a support ticket, find an answer in a manual, extract a date, or summarize a conversation.
- Each needs a different definition of success.

1.2 Understanding and generation

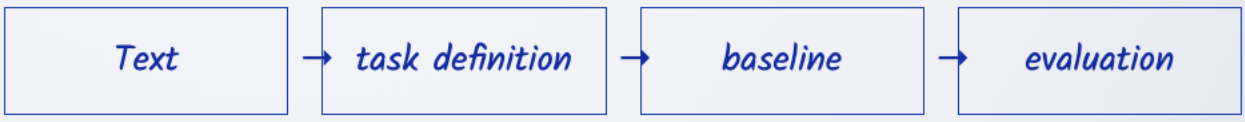
- Classification assigns labels.
- Extraction identifies spans.
- Retrieval finds relevant documents.
- Generation produces new text conditioned on input.

1.3 A practical baseline

- Collect representative examples.
- Reserve a test set.
- Try simple rules or a small model before adding a larger system.

1.4 Where it gets hard

- Ambiguity, domain jargon, sarcasm, multilingual input and changing data can all break an apparently strong demo.



2. Text preprocessing

Keep the information the task needs

2.1 Tokenization

- Split text using the tokenizer expected by your model.
- Words, subwords and bytes are different choices; token boundaries are model-dependent.

2.2 Normalize with care

- Whitespace cleanup can help.
- Lowercasing, punctuation removal and stop-word removal may erase useful meaning or violate a pretrained model's input conventions.

2.3 Stems and lemmas

- Stemming uses surface rules; lemmatization aims for dictionary forms.
- Neither is a mandatory step for transformer models.

2.4 Avoid leakage

- Fit vocabulary and preprocessing statistics on training data only.
- Apply the same saved pipeline to validation and test data.

"Not good" must not become "good".

3. Representing text

Choose what similarity should mean

3.1 Bag of words

- Count vocabulary items.
- It is inexpensive and interpretable, but ignores word order unless you add n-grams.

3.2 TF-IDF

- Increase the weight of terms distinctive to a document and reduce the influence of terms common across the collection.

3.3 N-grams

- Adjacent sequences preserve some local order.
- "Not useful" can behave differently from either word alone.

3.4 Dense vectors

- Learn continuous representations that can capture semantic relationships.
- Their quality depends on the training objective and the domain.

Counts → weighted counts → learned vectors

Hammad Yousuf

WITHHAMMAD

4. Word embeddings

Useful geometry, imperfect meaning

4.1 Static embeddings

- Word2Vec, GloVe and FastText learn reusable word representations.
- A static vector does not adapt to each new sentence.

4.2 Training intuition

- CBOW predicts a target from nearby words; skip-gram predicts nearby words from a target.
- These objectives learn distributional patterns.

4.3 Subword information

- FastText represents words through character n-grams, which can help with morphology and uncommon forms.

4.4 Similarity

- Cosine similarity compares vector directions.
- A high score is a model signal, not proof that two statements mean the same thing.

"bank" by a river ≠ "bank" holding a deposit

5. Language models

Predict a distribution over what comes next

5.1 The objective

- A causal language model estimates the next token from the preceding tokens.
- A sequence probability is built from these conditional predictions.

5.2 N-gram models

- Condition on a short fixed history.
- Smoothing prevents unseen combinations from receiving zero probability.

5.3 Neural models

- Learn reusable representations instead of keeping only explicit count tables.
- The architecture determines how context is processed.

5.4 Perplexity

- Measures how surprising held-out text is to the model.
- Compare scores only under compatible tokenization and evaluation conditions.

P(sequence) = product of next-token probabilities

6. Neural networks

From local patterns to sequence state

6.1 Feedforward networks

- Transform fixed-size features through learned layers.
- They are useful building blocks even inside large transformers.

6.2 Recurrent networks

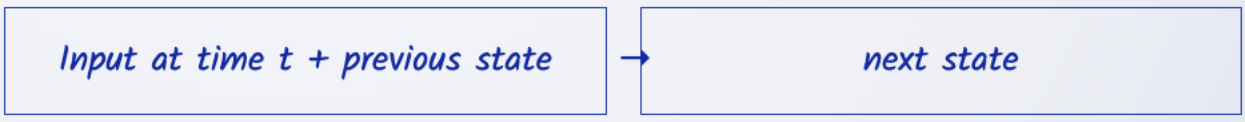
- Process a sequence through a changing hidden state.
- Long dependencies can be difficult to learn with simple recurrence.

6.3 LSTM and GRU

- Gates regulate which information to retain or update.
- These architectures help manage sequence information across steps.

6.4 Practical tradeoffs

- Choose based on latency, data size, sequence length and hardware.
- A newer architecture is not automatically the best fit for a small task.



Hammad Yousuf

WITHHAMMAD

7. Attention

Select relevant information from context

7.1 Queries, keys, values

- A query is compared with keys.
- Normalized scores weight the values, producing a context-dependent mixture.

7.2 Scaled dot product

- Compute QK^T, scale by $\sqrt{d_k}$, apply any mask, then softmax.
- Multiply those weights by V.

7.3 Self and cross attention

- Self-attention reads from the same sequence.
- Cross-attention uses queries from one sequence and keys and values from another.

7.4 Masks matter

- Causal masks block future positions.
- Padding masks exclude empty slots.
- A diagram without the mask can teach the wrong dependency.

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k} + \text{mask})V$$

8. The transformer

Attention plus repeated feature transformations

8.1 Input representation

- Token embeddings are combined with position information.
- Position can be represented in different ways across model families.

8.2 A repeated block

- Attention exchanges information between positions; a feedforward network transforms each position.
- Residual paths and normalization support the computation.

8.3 Architecture families

- Encoders are useful for representations.
- Decoders generate causally.
- Encoder-decoder models map an input sequence to an output sequence.

8.4 Parallel and sequential

- Prompt positions can be processed together within a layer.
- Ordinary autoregressive output still depends on previously generated tokens.



9. Pretrained models

Match the model to the job

9.1 BERT-style encoders

- Masked-language pretraining develops contextual representations.
- Add an appropriate head for classification or extraction.

9.2 GPT-style decoders

- Causal pretraining learns next-token prediction.
- Instruction tuning can adapt that base capability to following requests.

9.3 TS-style models

- Express many tasks as text-to-text transformations with an encoder-decoder architecture.

9.4 Selection checklist

- Check license, language coverage, context length, hardware, latency and task evaluations.
- Parameter count alone does not decide quality.



10. Fine-tuning

Change behavior with targeted training

10.1 When it helps

- Adapt style, output structure or a repeatable task when prompting and retrieval do not meet measured requirements.

10.2 Full fine-tuning

- Update the model's trainable weights.
- This can require substantial memory and careful optimization.

10.3 LoRA adapters

- Train small low-rank updates to selected layers.
- This reduces trainable parameter count; it does not make data quality optional.

10.4 Evaluation discipline

- Keep a held-out set and compare with the original model.
- Check regression on capabilities you still need.



11. NLP applications

The same text can support different tasks

11.1 Classification and routing

- Map a message to a category or intent.
- Track per-class errors and the consequences of a wrong label.

11.2 Entity extraction

- Find names, dates or domain-specific fields.
- Evaluate boundaries as well as the extracted values.

11.3 Translation and summaries

- Preserve meaning and important qualifications.
- A fluent answer may still omit or invent details.

11.4 Question answering

- Separate the ability to retrieve relevant evidence from the ability to answer accurately using it.

Choose the task before choosing the model.

12. Generating text

Decoding changes the output

12.1 Greedy decoding

- Choose the highest-scoring next token each step.
- It is simple but can produce repetitive continuations.

12.2 Sampling

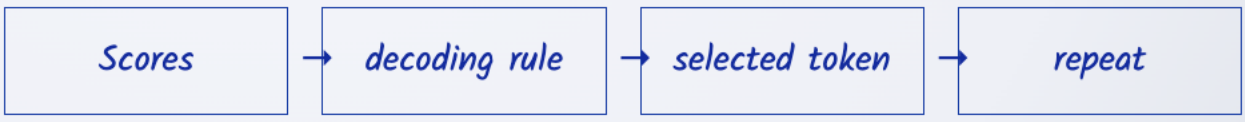
- Draw from a probability distribution.
- Temperature rescales logits; top-k and top-p restrict the candidate pool.

12.3 Beam search

- Track several candidate sequences.
- It can be useful for some constrained tasks, but is not a universal creativity setting.

12.4 Control and verify

- Use stop conditions, length limits and schema validation.
- A low temperature does not guarantee truth or perfect reproducibility.



Hammad Yousuf
WITHHAMMAD

13. Retrieval and search

Find the evidence before drafting the answer

13.1 Lexical retrieval

- BM25 rewards matching terms while accounting for term frequency and document length.
- Exact identifiers often benefit from lexical search.

13.2 Dense retrieval

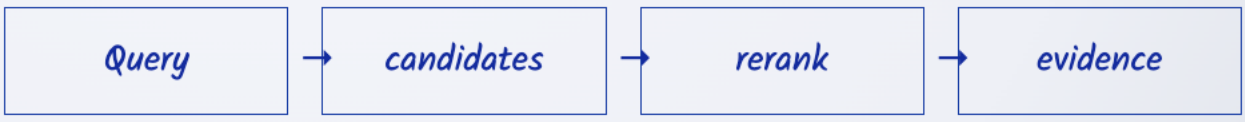
- Embed a query and documents into a compatible vector space, then retrieve by vector similarity.

13.3 Hybrid search

- Combine lexical and dense results.
- A reranker can score a smaller candidate set with more expensive relevance judgments.

13.4 Measure retrieval

- Use labeled queries and relevance judgments.
- Recall@k and ranking metrics reveal failures an answer-only demo can hide.



14. Retrieval-augmented generation

Ground answers in selected documents

14.1 Index preparation

- Parse source documents, preserve metadata and permissions, split into meaningful chunks, and index the result.

14.2 At query time

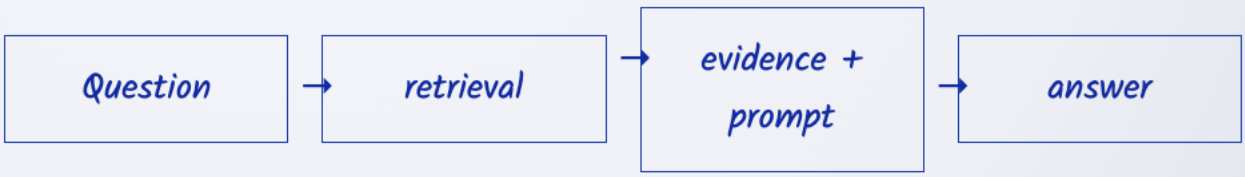
- Retrieve candidate chunks, rerank if useful, then supply selected evidence and clear answer instructions to the model.

14.3 Citations need checking

- RAG can support citations, but it does not guarantee that an answer is true or that a cited passage supports it.

14.4 Failure analysis

- Inspect whether the right document was available, retrieved, included in context and correctly used.
- Fix the failing stage.



15. Evaluating NLP

Measure the behavior you intend to ship

15.1 Classification

- Precision asks how many predicted positives were correct; recall asks how many true positives were found.
- F1 balances both.

15.2 Generation metrics

- BLEU and ROUGE measure forms of reference overlap.
- Semantic metrics add another signal, but neither replaces task-specific review.

15.3 Human review

- Use a clear rubric for correctness, completeness and usefulness.
- Review disagreements instead of averaging them away.

15.4 Slices and baselines

- Evaluate rare cases, languages and difficult inputs separately.
- Compare with a stable baseline and report sample size.

A score without a task is hard to interpret.

16. Deployment and MLOps

Ship the whole system, then observe it

16.1 Version the pipeline

- Track the model, tokenizer, prompt, retrieval index and evaluation set.
- A model version alone does not reproduce an application.

16.2 Serve predictably

- Set timeouts, concurrency limits and response validation.
- Measure latency distributions, throughput and cost.

16.3 Monitor drift

- Watch changing input patterns and real failure reports.
- Maintain a rollback path and refresh evaluation examples.

16.4 Protect access

- Apply authentication, authorization and data boundaries in code.
- Model instructions are not a substitute for access controls.



17. Advanced patterns

Add complexity for a measured reason

17.1 Long context

- More input capacity can help, but relevance, latency and retrieval quality still matter.
- Test whether the extra context improves the task.

17.2 Multimodal systems

- Text can be combined with image, audio or video representations.
- Evaluate each input channel and their interactions.

17.3 Tool use and agents

- A model proposes actions; an application executes permitted operations and feeds back results.
- Keep stopping rules and budgets explicit.

17.4 Efficient inference

- Caching, batching and quantization target different costs.
- Benchmark quality and latency on the actual workload.

Add a component only when it earns its place.

18. Tools and frameworks

Build a small, understandable stack

18.1 Text and baselines

- Use standard Python processing and scikit-learn for count features, TF-IDF and classical models.

18.2 Neural models

- PyTorch supports model development; Hugging Face Transformers and Datasets provide model and data tooling.

18.3 Search infrastructure

- Choose an index that fits your metadata, filtering, scale and operational constraints.
- Keep ingestion and querying testable.

18.4 Orchestration

- Frameworks can simplify pipelines, but make inputs, outputs, retries and permissions visible.
- Keep provider-specific code isolated.



19. Build your first project

Turn these notes into evidence

19.1 A useful project

- Build a support-document assistant.
- Start with a small set of documents and twenty realistic questions, including unanswerable ones.

19.2 The first version

- Implement retrieval, an evidence-bound answer prompt and citations.
- Log the retrieved passages and final answer for review.

19.3 The next experiment

- Compare lexical, dense and hybrid retrieval.
- Change one component at a time and rerun the same evaluation set.

19.4 Explain your decisions

- Be ready to justify tokenization, chunk size, model choice, access checks and failure handling.
- Show errors as well as successes.

Save the notebook. Build one thing you can test.

Hammad Yousuf

WITHHAMMAD